

## Actividad | # 3 | Programa 2 (parte 2)

### Desarrollo de Aplicaciones Móviles

#### IV

---

Ingeniería en Desarrollo de  
Software



TUTOR: Marco Alonso Rodriguez

ALUMNO: Casandra Montserrat Ortiz Cortes

FECHA:01/08/2026

## Índice

Introducción.....1

Descripción.....2

Justificación.....3

Desarrollo.....4

o Codificación

o Prueba del programa

Conclusión.....5

Referencia

## INTRODUCCION

En la práctica tiene como finalidad aplicar los conocimientos básicos del lenguaje de programación Swift mediante el desarrollo de un programa que simula el funcionamiento de un cajero automático. Durante la elaboración del código se utilizaron comandos simples como la declaración de variables, funciones, estructuras de control if, switch y while, además del uso de `readLine()` para recibir datos del usuario desde la consola. El programa permite realizar operaciones sencillas como depósitos, retiros, consultar el saldo disponible y salir del sistema. También se implementaron validaciones para evitar errores al ingresar datos y verificar que exista saldo suficiente antes de realizar un retiro. Con esta práctica reforcé la importancia de la lógica de programación y la organización del código mediante funciones, lo que facilita su comprensión, mantenimiento y reutilización. Además, esta actividad permitió fortalecer mis habilidades para resolver problemas y comprender mejor el funcionamiento de programas interactivos en Swift de manera.

## DESCRIPCION

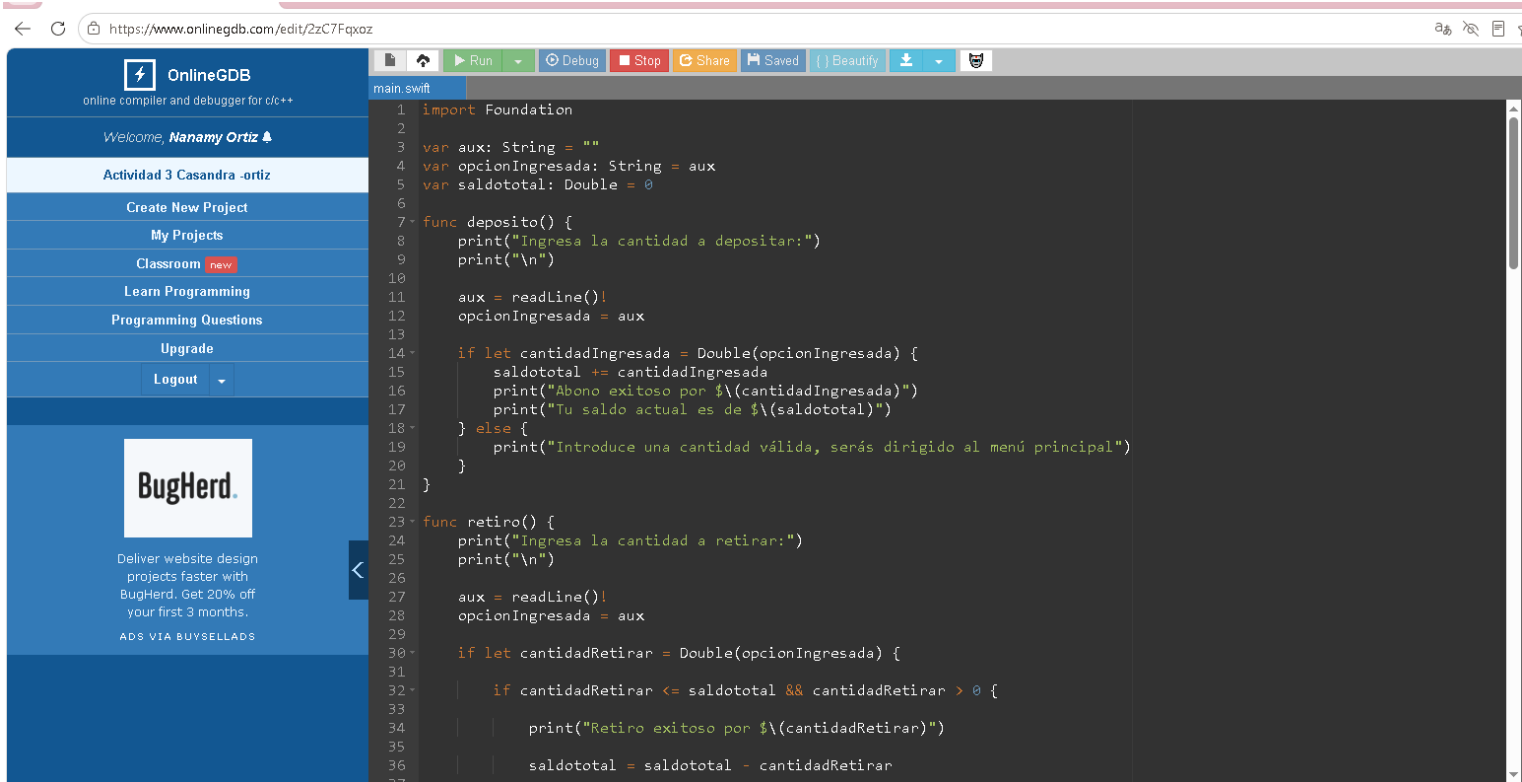
Este programa fue diseñado para representar el funcionamiento básico de un cajero automático mediante el uso de instrucciones sencillas en el lenguaje Swift. Su estructura permite al usuario interactuar con un menú de opciones para realizar depósitos, retiros, consultar el saldo disponible o finalizar la ejecución. Cada operación se encuentra organizada en funciones independientes, facilitando la lectura y comprensión del código. Además, se emplean estructuras de decisión para validar las acciones del usuario y ciclos que mantienen activo el menú hasta que se seleccione la opción de salida. También se realizan comprobaciones para evitar depósitos inválidos o retiros superiores al saldo disponible. El uso de variables permite almacenar temporalmente la información ingresada y actualizar el saldo después de cada movimiento. En conjunto, el programa demuestra la aplicación de conceptos fundamentales de programación, fortaleciendo la práctica del desarrollo de soluciones simples, organizadas y funcionales para resolver problemas de forma eficiente correctamente.

## JUSTIFICACION

En esto se explica la importancia de desarrollar este programa como parte del aprendizaje del lenguaje Swift. Mediante esta práctica fue posible aplicar los conocimientos adquiridos sobre variables, funciones, estructuras de control y validación de datos. Además, permitió comprender cómo organizar un programa para que sea más claro, ordenado y fácil de mantener. El desarrollo de este proyecto también fortaleció la capacidad para analizar problemas y proponer soluciones utilizando lógica de programación. Asimismo, ayudó a mejorar la comprensión del funcionamiento de los menús interactivos y la manipulación de información ingresada por el usuario. Gracias a esta experiencia fue posible reforzar habilidades prácticas que serán útiles en futuros proyectos académicos y profesionales. Finalmente, esta actividad demuestra la importancia de practicar constantemente para desarrollar programas funcionales, eficientes y bien estructurados que respondan correctamente a las necesidades del usuario final con responsabilidad, dedicación y compromiso durante mi formación académica continua como estudiante universitario.

# DESARROLLO

## o Codificación



The screenshot displays the OnlineGDB web interface. The left sidebar contains navigation links: 'Welcome, Nanamy Ortiz', 'Actividad 3 Casandra -ortiz', 'Create New Project', 'My Projects', 'Classroom' (marked as new), 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main editor area shows Swift code for a banking application. The code includes imports, variable declarations, and functions for depositing and withdrawing money.

```
1 import Foundation
2
3 var aux: String = ""
4 var opcionIngresada: String = aux
5 var saldototal: Double = 0
6
7 func deposito() {
8     print("Ingresa la cantidad a depositar:")
9     print("\n")
10
11     aux = readLine()!
12     opcionIngresada = aux
13
14     if let cantidadIngresada = Double(opcionIngresada) {
15         saldototal += cantidadIngresada
16         print("Abono exitoso por $\(cantidadIngresada)")
17         print("Tu saldo actual es de $\(saldototal)")
18     } else {
19         print("Introduce una cantidad válida, serás dirigido al menú principal")
20     }
21 }
22
23 func retiro() {
24     print("Ingresa la cantidad a retirar:")
25     print("\n")
26
27     aux = readLine()!
28     opcionIngresada = aux
29
30     if let cantidadRetirar = Double(opcionIngresada) {
31
32         if cantidadRetirar <= saldototal && cantidadRetirar > 0 {
33
34             print("Retiro exitoso por $\(cantidadRetirar)")
35
36             saldototal = saldototal - cantidadRetirar
37         }
38     }
```

OnlineGDB

online compiler and debugger for c/c++

Welcome, **Nanamy Ortiz**

Actividad 3 Casandra -ortiz

Create New Project

My Projects

Classroom **new**

Learn Programming

Programming Questions

Upgrade

Logout



Deliver website design projects faster with BugHerd. Get 20% off your first 3 months.

ADS VIA BUYSPELLADS

main.swift

```
37
38     print("Saldo actual: ${saldototal}")
39
40 } else {
41
42     print("Lo siento, la cantidad que quieres retirar excede el límite de saldo disponible")
43 }
44
45 } else {
46
47     print("Cantidad no válida")
48 }
49 }
50
51 while opcionIngresada != "4" {
52
53     print("\n")
54     print("***** BANCO COPPEL *****")
55     print("\n")
56     print("1.- Depósito")
57     print("2.- Retiro")
58     print("3.- Consultar saldo")
59     print("4.- Salir")
60
61     aux = readline()!
62     opcionIngresada = aux
63
64     switch opcionIngresada {
65
66     case "1":
67
68         deposito()
69
70         print("\n")
71         print("¿Deseas realizar otro depósito? (S/N): ")
72
73         aux = readline()!
74         opcionIngresada = aux
75     }
```

OnlineGDB

online compiler and debugger for c/c++

Welcome, **Nanamy Ortiz**

Actividad 3 Casandra -ortiz

Create New Project

My Projects

Classroom **new**

Learn Programming

Programming Questions

Upgrade

Logout

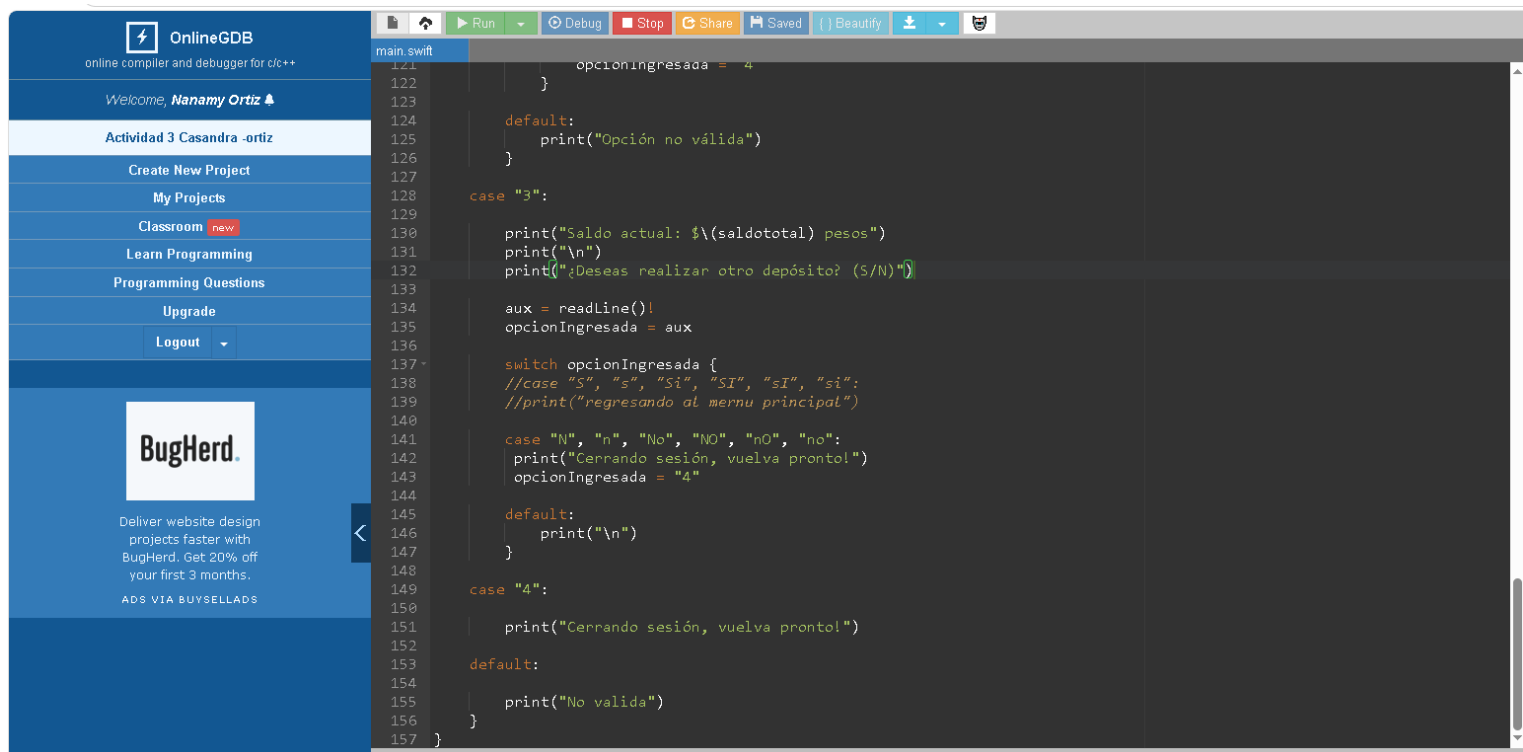


Deliver website design projects faster with BugHerd. Get 20% off your first 3 months.

ADS VIA BUYSPELLADS

main.swift

```
76 switch opcionIngresada {
77
78     case "S":
79         deposito()
80
81     case "N":
82
83         print("¿Deseas continuar con otra operación? (N/S)")
84
85         aux = readline()!
86         opcionIngresada = aux
87
88         if opcionIngresada == "N" {
89             print("Cerrando sesión, vuelva pronto")
90             opcionIngresada = "4"
91         }
92
93     default:
94         print("\n")
95     }
96
97     case "2":
98
99         retiro()
100
101         print("\n")
102         print("¿Deseas realizar otro retiro? (S/N): ")
103
104         aux = readline()!
105         opcionIngresada = aux
106
107         switch opcionIngresada {
108
109         case "s", "S", "Si", "SI", "sI", "sI":
110             retiro()
111         }
```

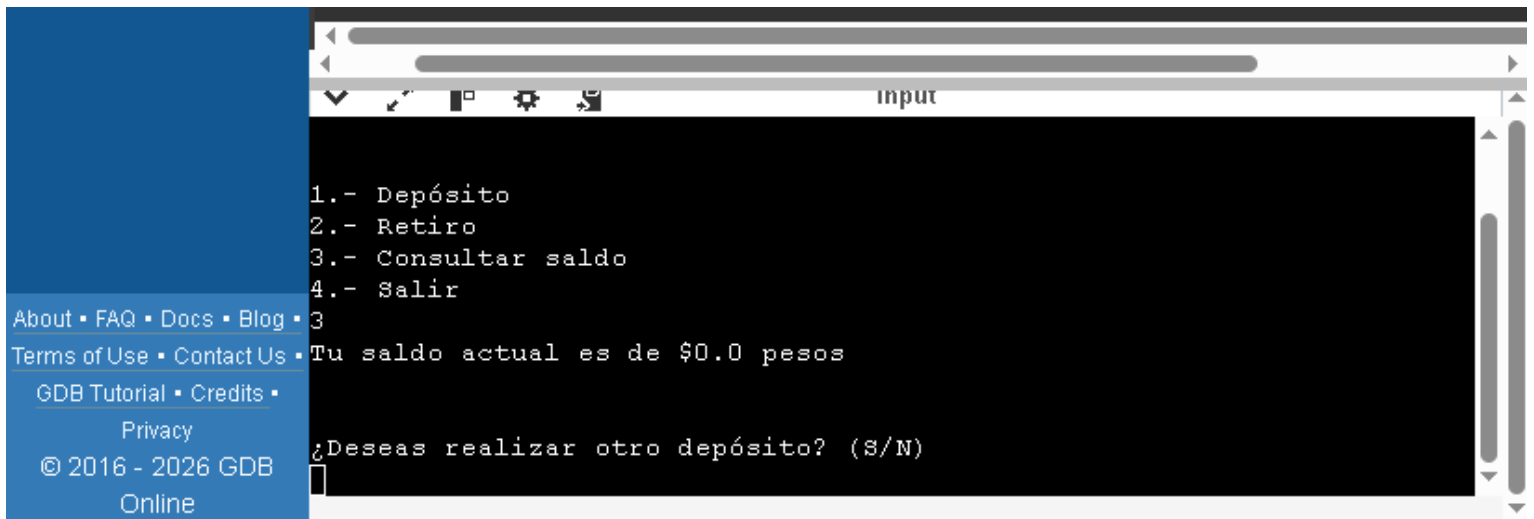


The screenshot displays the OnlineGDB web interface. On the left, a sidebar contains navigation links: 'Welcome, Nanamy Ortiz', 'Actividad 3 Casandra -ortiz', 'Create New Project', 'My Projects', 'Classroom new', 'Learn Programming', 'Programming Questions', 'Upgrade', and 'Logout'. The main area shows a Swift code editor with the following code:

```
121 | }
122 | } opcionIngresada = 4
123 |
124 | default:
125 |     print("Opción no válida")
126 | }
127 |
128 | case "3":
129 |
130 |     print("Saldo actual: ${saldototal} pesos")
131 |     print("\n")
132 |     print("¿Deseas realizar otro depósito? (S/N)")
133 |
134 |     aux = readLine()!
135 |     opcionIngresada = aux
136 |
137 |     switch opcionIngresada {
138 |     //case "S", "s", "Si", "SI", "sI", "si":
139 |     //print("regresando al menu principal")
140 |
141 |     case "N", "n", "No", "NO", "nO", "no":
142 |         print("Cerrando sesión, vuelva pronto!")
143 |         opcionIngresada = "4"
144 |
145 |     default:
146 |         print("\n")
147 |     }
148 |
149 | case "4":
150 |
151 |     print("Cerrando sesión, vuelva pronto!")
152 |
153 | default:
154 |
155 |     print("No valida")
156 | }
157 | }
```

Las imágenes muestran el desarrollo del programa en el compilador OnlineGDB, utilizando el lenguaje de programación Swift. En ellas se observa la creación de variables para almacenar la información del usuario y el saldo disponible, así como las funciones encargadas de realizar depósitos y retiros. También se puede apreciar la implementación de un menú principal mediante la estructura switch, el cual permite seleccionar las diferentes opciones disponibles, como depositar dinero, retirar efectivo, consultar el saldo y salir del programa. Además, se utilizaron estructuras de control como if y while para validar la información ingresada y mantener el funcionamiento del menú hasta que el usuario decida finalizar la ejecución. Durante el desarrollo se realizaron ajustes y correcciones para mejorar el funcionamiento del programa, logrando un código más ordenado, comprensible y funcional, aplicando los conocimientos adquiridos durante la práctica de programación en Swift.

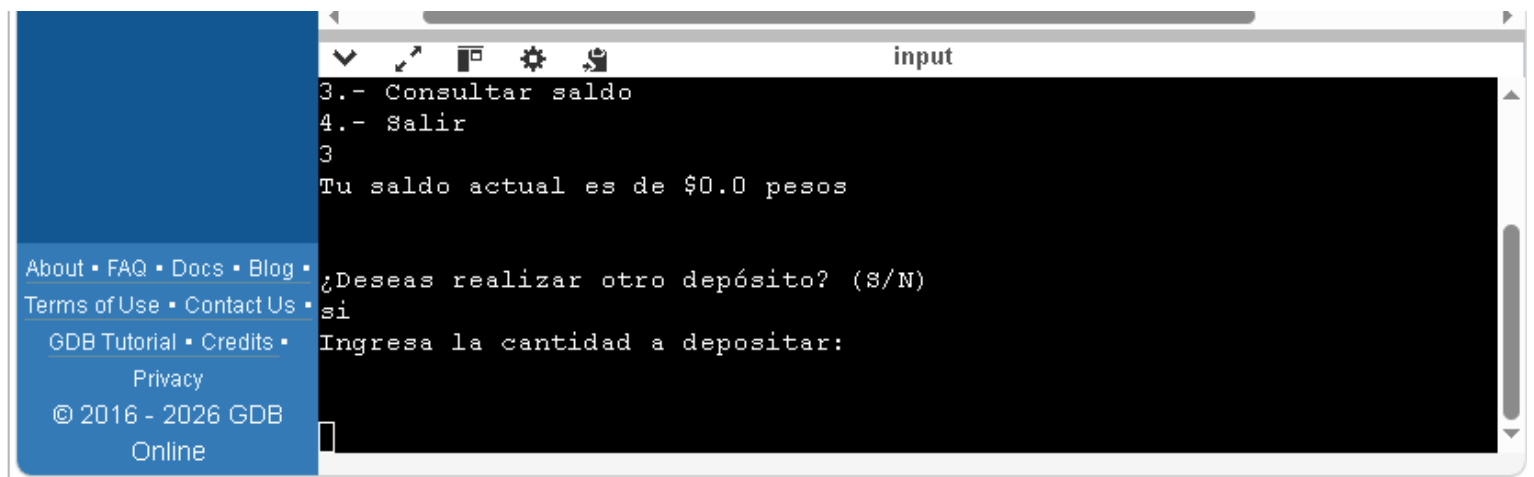
## ○ Prueba del programa



```
1.- Depósito
2.- Retiro
3.- Consultar saldo
4.- Salir
3
Tu saldo actual es de $0.0 pesos
¿Deseas realizar otro depósito? (S/N)
```

### Consulta de saldo

En esta imagen se muestra la ejecución del programa al seleccionar la opción 3, Consultar saldo. El sistema despliega el saldo disponible, que en ese momento es de \$0.0 pesos, y posteriormente pregunta al usuario si desea realizar otro depósito, permitiendo continuar con la operación.



```
3.- Consultar saldo
4.- Salir
3
Tu saldo actual es de $0.0 pesos
¿Deseas realizar otro depósito? (S/N)
si
Ingresa la cantidad a depositar:
```

### Solicitud de depósito

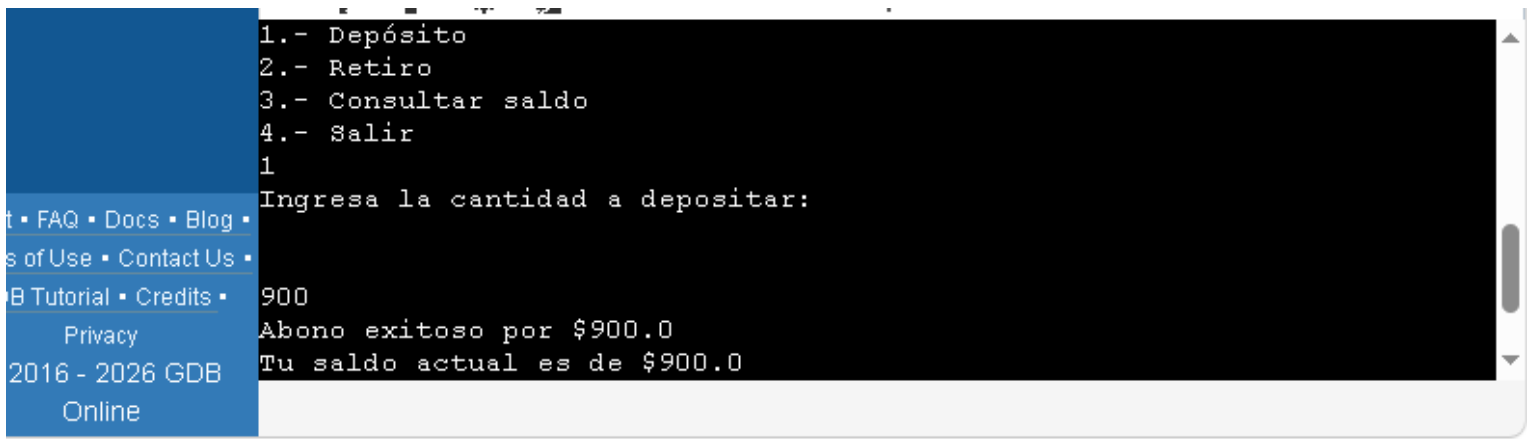
En esta captura el usuario responde que sí desea realizar un depósito. El programa solicita ingresar la cantidad de dinero que se desea depositar para actualizar el saldo de la cuenta.





### Menú principal

En esta imagen se observa el menú principal del programa después de completar una operación. El sistema vuelve a mostrar las opciones disponibles: depósito, retiro, consulta de saldo y salir, permitiendo que el usuario continúe utilizando el cajero.



### Depósito realizado

Esta captura muestra el resultado de un depósito exitoso por \$900.00. El programa confirma la operación y actualiza el saldo disponible, mostrando que el nuevo saldo es de \$900.00.



```
input
Tu saldo actual es de $900.0

¿Deseas realizar otro depósito? (S/N):
no

**** BANCO COPPEL ****

4.- Salir
2
```

### Regreso al menú principal

En esta imagen se observa que, después de realizar un depósito y mostrar el saldo actualizado de \$900.00, el programa pregunta al usuario si desea efectuar otro depósito. Al responder "no", el sistema regresa automáticamente al menú principal, donde nuevamente presenta las opciones disponibles para realizar otra operación o finalizar la ejecución del programa. Esto permite que el usuario continúe utilizando el sistema sin necesidad de reiniciar la aplicación.



```
input
4.- Salir
2
Ingresa la cantidad a retirar:
360
Retiro exitoso por $360.0
Saldo actual: $540.0

¿Deseas realizar otro retiro? (S/N):

```

### Retiro realizado

En esta imagen se observa un retiro exitoso por \$360.00. El sistema descuenta la cantidad del saldo disponible, muestra el nuevo saldo de \$540.00 y pregunta al usuario si desea realizar otro retiro.

Link zip : <https://github.com/casandraortiz31/mi-Proyecto/blob/9814224c4c691939b0718bf28843234c4ddc9f0e/Casandra%20Ortiz%20Act-3.zip>